Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrare practica Nr. 6

Disciplina: Tehnici de programare

Tema: ” Informații invizibile:forensics și steganografie ”

A efectuat : st. gr.SI – 263 -Boicu Ecaterina

A verificat : asistent universitar - Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

Să se analizeze fișiere digitale folosind tehnici de decodare , analiza forensica și steganografie. Lucrarea cuprinde doua parti:
realizarea unui decoder care recunoaste si desface codari succesive , identifica un mesaj criptat prin XOR și gaseste parole prin compararea hash-urilor cu o lista de cuvinte ; investigarea fișierelor pentru identificarea tipului real , extragerea textului și a metadatelor , recuperarea datelor ascunse și ascunderea sau dezvaluirea unui mesaj într-o imagine prin metoda LSB.

SCOPUL LUCRĂRII

Scopul lucrării numărul 6 este de a înțelege cum pot fi detectate și recuperate informații care nu sunt vizibile la o examinare obișnuită a fișierelor . Pin itemii propuși , se urmărește însușirea tehnicilor de decodare a straturilor hex și Base64 , de analiza a octetilor si metadatelor , de recuperare a fișierelor atașate (legate) la alte fișiere și de utilizare a biților cei mai puțin semnificativi pentru steganografie.

ETAPELE LUCRĂRII

Împărțirea lucrării de laborator in doua parti: Lucrarea A - un decoder universal de straturi de codare , XOR și hash-uri ; Lucrarea B - analiza forensica de fisiere si steganografie prin biții cei mai puțin semnificativi (LSB)- Last Significant Bit.

Ce predai și cum se notează

L	i	v	r	a
`arsenal/forensics/decode.py`	Decoderul: recunoastere, desfacere, straturi, XOR brute, hash (A2-A7).			
`arsenal/forensics/fisa.json`	Fisa unui mesaj decodat, salvata de unealta (A7).			
`teste/test_decode.py`	Testul care trece (A7).			
forensics.py` (sau similar)	Scriptul cu B1-B4: magic bytes, strings, EXIF/GPS, carving.			
stego.png` + scriptul LSB	Poza cu mesaj ascuns si codul care ascunde/ dezvaluie (B5-B6).			
raport.md` (secțiune nouă)	Ce ai decodat, ce cheie/parola ai spart, metadate GPS, arhiva ascunsa, mesajul LSB.			

Fig.1 - Tabelul de implementare a pașilor de lucru

PAȘII DE LUCRU

Pregatirea fișierelor de analizat: Au fost create fișiere de probă care conțin texte codate , date criptate prin XOR , hash-uri , imagini cu metadate și conținut suplimentar ascuns.

Realizarea decoderului universal . S-a implementat recunoașterea codarilor URL , hex si Base64 , urmată de decodarea repetata a straturilor pana la obținerea textului. Pentru mesajul analizat , straturile au fost BAse64 si hex , iar rezultatul a fost FLAG{straturi_de_encoding}

Analiza XOR si a hash-urilor . Au fost încercate cheile posibile pentru XOR pe un octet , iar cheia corecta identificată a fost 128. Mesajul recuperat a fost FLAG{xor_un_octet}.Pentru hash-uri,s-a folosit o lista de cuvinte și s-a identificat parola password , folosind algoritmul MD5.

Verificarea tipului real și căutarea textului în fișiere . Tipul fișierelor a fost determinat pe baza semnăturilor binare , nu doar a extensiilor. Cautarea sirurilor imprimabile in imaginea PNG a scos la iveala indicii despre un fisier ascuns.

extragerea metadatelor și recuperarea arhivei . Din fotografia analizată , au fost citite metadatele EXIF , inclusiv coordonatele GPS 47.016667, 28.857500 , corespunzătoare orașului Chișinău . In fisierul PNG a fost găsită și extrasa o arhiva ZIP , din care s-a recuperat mesajul FLAG {ascuns_prin_craving}.

Aplicarea steganografiei LSB. Un mesaj a fost ascuns în ultimii biți ai canalelor RGB ale pixelilor, apoi extras prin parcurgerea paginii în aceeași ordine . Mesajul recuperat FLAG{ascuns_in_pixeli}, a fost identic cu cel introdus.

Rezolvarea exercitiului bonus și verificarea rezultatelor. lanțul de decodare Base64 și XOR a fost desfăcut , rezultand FLAG{lant_complet_despicat}. Testele automate pentru decoder au confirmat funcționarea operatiilor implementate.

MEDIUL DE LUCRU ȘI STRUCTURA PROIECTULUI

Lucrarea a fost realizată în Python 3.9.6 , folosind biblioteca Pillow(pip3 install pillow) pentru prelucrarea imaginilor.Structura proiectului:

```
lab6/
├── probe/
│   ├── pregateste.py       (genereaza fisierele-tinta, rulat o singura data)
│   └── (fisierele generate: mesaj.txt, xor.bin, foto.jpg, ...)
├── arsenal/forensics/
│   ├── decode.py          Lucrarea A - decoderul universal
│   └── fisa.json          rezultatul salvat automat de decoder
├── teste/
│   └── test_decode.py      6 teste automate (pytest)
├── forensics.py           Lucrarea B - forensics si steganografie
```

Fig.2.1- Structura proiectului

```
├── stego.png           poza cu mesajul ascuns prin LSB
├── gasit.zip           arhiva extrasa prin file carving
└── raport.md           rezumatul rezultatelor
```

Fig.2.2 - Structura proiectului

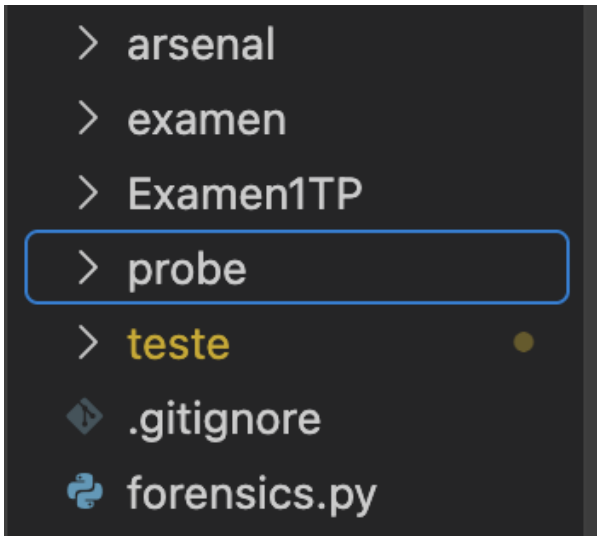


Fig.3 - Fișierele create in VS Code

I. Lucrarea A - Decoderul universal

Obiectivul acestei părți este constituirea unui instrument de linie de comanda capabil să recunoască automat tipul de codare al unui text , sa il desfaca , si sa repete procesul atunci cand rezultatul este , la randul lui , un alt strat de codare . Separat , instrumentul oferă și două module de atac prin forța brută : spargerea unei cifrari XOR pe un octet si spargerea parolelor hash-uite prin dictionar .

1.1. Recunoașterea tipului de codare

Fiecare tip de codare are o forma caracteristica , posibil de identificat cu expresii regulate : codare URL - conține secvențe de forma %xx (procent urmat de doua cifre hexazecimale - baza 16) ; hexazecimal folosește exclusiv cifrele 0-9 și literele a-f/A-F , cu o lungime para : base64 folosește alfabetul A-Z ,a-z ,0-9,+ și / , eventual cu adding = la final , cu o lungime multiplu de 4. Ordinea verificarii conteaza deoarece un șir hexazecimal valid se poate încadra și în alfabetul base64 , verificarea pentru hex trebuie sa meargă înainte pentru verificarea base64.

```
18
19  def ghici_strat(s):
20      # URL-encoded: contine secvente %xx (doua cifre hex dupa %)
21      if re.search(r"%[0-9A-Fa-f]{2}", s):
22          return "url"
23      # Hex: doar 0-9 a-f A-F, lungime para, si nevid
24      if len(s) > 0 and len(s) % 2 == 0 and re.fullmatch(r"[0-9A-Fa-
25          return "hex"
26      # Base64: alfabetul standard, lungime multiplu de 4 (cu paddin
27      if len(s) > 0 and len(s) % 4 == 0 and re.fullmatch(r"[A-Za-z0-
28          return "base64"
29      return "necunoscut"
30
31
32  print(ghici_strat("%2Fetc"))        # ar trebui: url
33  print(ghici_strat("48656c6c6f"))    # ar trebui: hex
34  print(ghici_strat("U2FsdXQ="))      # ar trebui: base64
```

```
Problems  Output  Terminal  ···                    Python

ekaterinalungul@MacBook-Air-Ekaterina Tehnici de Programare % /usr/bin/pytho
s/
ekaterinalungul/Documents/Tehnici de Programare/forensics/decode.py"
hex: 53616c7574
inapoi din hex: b'Salut'
ekaterinalungul@MacBook-Air-Ekaterina Tehnici de Programare % /usr/bin/pytho
s/ekaterinalungul/Documents/Tehnici de Programare/forensics/decode.py"
b'Salut' 83
hex: 53616c7574
inapoi din hex: b'Salut'
url
hex
base64
ekaterinalungul@MacBook-Air-Ekaterina Tehnici de Programare %
```

Fig.1.3 - Exemplu de recunoaștere a tipului / stratului de codare

1.2. Desfacerea straturilor multiple

Fișierul mesaj.txt conține un mesaj codat pe trei straturi suprapuse: base64 → base 64→ hex. Algoritmul de desfacere repeta operația de decodare atat timp cat formă rezultatului mai este recunoscută ca un tip de codare cunoscut .

Nota: Criteriul de oprire s=bazat strict pe “text imprimabil” este insuficient : fiecare strat intermediar (un șir hexazecimal , sau un alt base64) este, la randul sau , format din caractere imprimabile/ editabile . Oprirea prematură la primul rezultat imprimabil ar fi scos doar un singur strat , lasand mesajul încă parțial codat. Criteriul corect de oprire folosit este: decodarea continua cat timp tipul de codare mai este recunoscut și se oprește doar atunci când forma rezultatului nu mai corespunde niciunui tip cunoscut- moment în care rezultatul este textul clar final.

```
60  def despica(val, max_incercari=8, afiseaza=False):
61      """
62      Decodeaza repetat 'val' cat timp forma lui mai este recunoscuta ca un
63      strat de codare (ghici_strat != "necunoscut"), pana nu mai e nimic de
64      desfacut (text clar) sau se epuizeaza incercarile.
65      """
66      straturi = []
67      cur = val
68      b = cur.encode() if isinstance(cur, str) else cur
69      for _ in range(max_incercari):
70          strat, b = desfa(cur)
71          if strat == "necunoscut":
72              break
73          straturi.append(strat)
74          try:
75              cur = b.decode()
76          except UnicodeDecodeError:
77              break
78      if afiseaza:
79          print("straturi:", " -> ".join(straturi) if straturi else "(niciunul)")
80          try:
81              print("rezultat:", b.decode())
82          except UnicodeDecodeError:
83              print("rezultat (hex):", b.hex())
84      return straturi, b
85
86  val = open("probe/mesaj.txt").read().strip()
87  despica(val, afiseaza=True)
```

```
Problems  Output  Debug Console  Terminal  Ports                    Python

ekaterinalungul@MacBook-Air-Ekaterina Tehnici de Programare % clear
ekaterinalungul@MacBook-Air-Ekaterina Tehnici de Programare % /usr/bin/python3 "/User
s/ekaterinalungul/Documents/Tehnici de Programare/forensics/decode.py"
b'Salut' 83
hex: 53616c7574
inapoi din hex: b'Salut'
url
hex
base64
('base64', b'Salut')
('hex', b'Hello')
straturi: base64 -> base64 -> hex
rezultat: FLAG{straturi de encoding}
```

Fig.1.4 - Rezultatul obtinut pentru mesaj.txt

1.3 Spargerea cifrarii XOR pe un octet

Operatia XOR este reversibila : daca un text ⊕ cheie = text_criptat , atunci text-criptat ⊕ cheie = text din nou. Pentru o cheie formată dintr-un singur octet , exista doar 256 de valori posibile (0-225) , ceea ce permite o verificare deplina (forța brută) : se încearca fiecare cheie , iar rezultatele sunt retinute doar daca toți octeții decodati se încadrează în intervalul imprimabil ASCII (32-126). Pe fișierul xor.bin , cateva chei produc , din intamplare , octeți tehnic imprimabili , dar o singura cheie produce un text cu sens coerent :

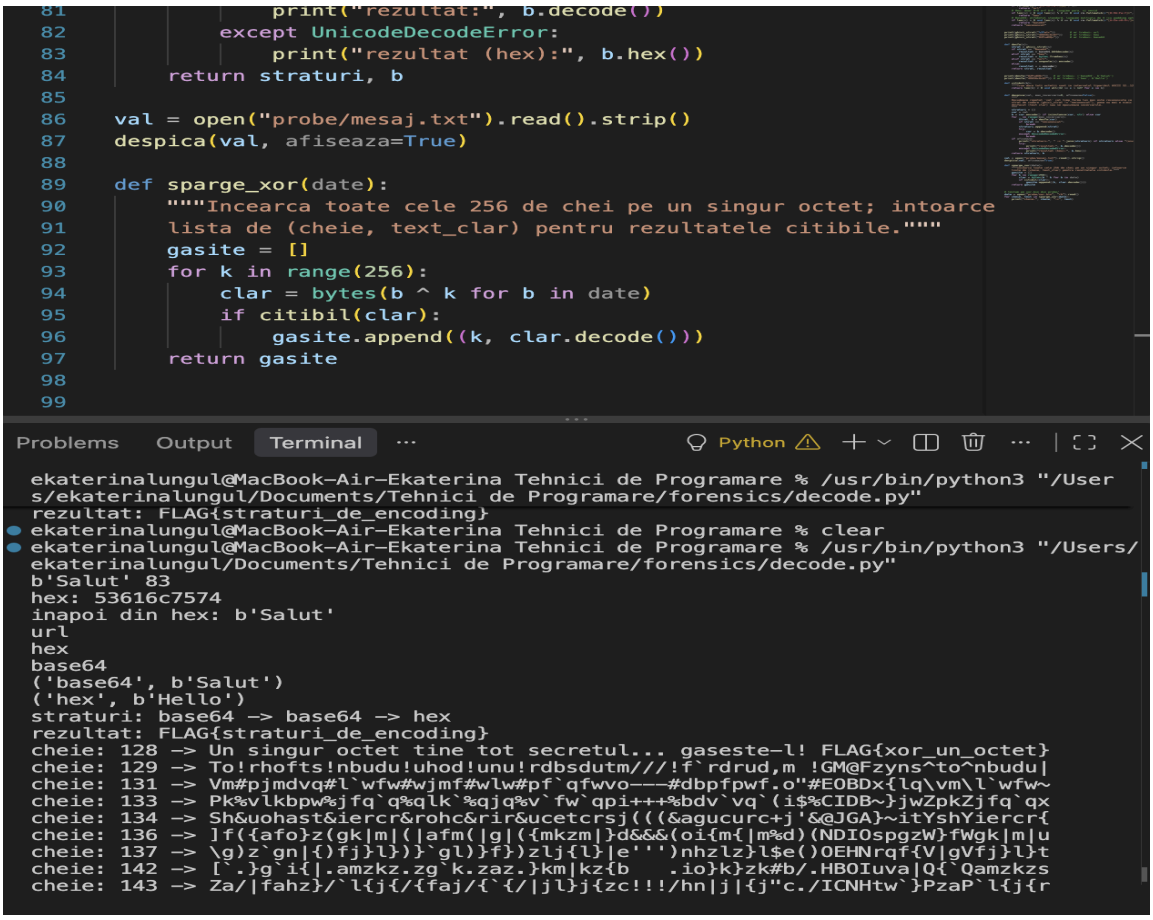


Fig.1.5 - Identificarea cheii 128 → “Un singur octet tine tot secretul…gaseste-l”

1.4 Spargerea unei parole hash-uite prin dictionar

O functie hash criptografica (MD5,SHA1,SHA256) nu poate fi inversată direct , însă , dacă se presupune parola originală , se poate calcula hash-ul ei și compara cu țintă . Lungimea hash-ului în caractere hexazecimale indica algoritmul folosit : 32 de caractere pentru MD5, 40 pentru SHA1 , 64 pentru SHA256. Fiecare cuvant din wordlist.txt este încercat pe rand , pana la găsirea potrivirii.



Fig.1.6 - Parola identificată

1.5 Interfata de linie de comanda

Toate activitățile / funcțiile descrise precedent sunt expuse printr-un instrument de linie d comanda (argparse) ,cu trei moduri de utilizare .

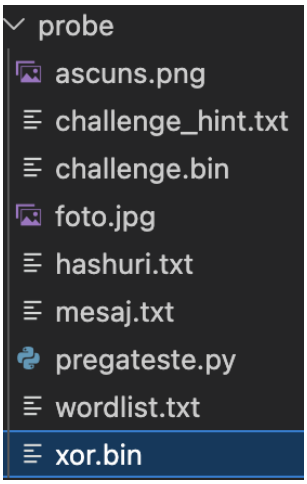


Fig.1.6.1 - Fișierul “probe” cu file-uri generate

```
# desfacere de straturi, direct pe un fisier
python3 arsenal/forensics/decode.py probe/mesaj.txt --fisier

# spargere XOR pe un octet
python3 arsenal/forensics/decode.py probe/xor.bin --fisier --xor

# spargere hash prin dictionar
python3 arsenal/forensics/decode.py "<hash>" --hash
```

Fig.1.6.2 - Liniile de comanda explicite

1.6 Testarea automată

Corectitudinea decoderului a fost verificată prin șase teste automate (pytest), care acoperă : decodarea corecta pentru base64,hex. și URL-encoding ; recunoașterea unui text necodat ca fiind “necunoscut”; desfacerea completa a unui lanț de trei straturi ; și spargerea corecta a cifrarii XOR .

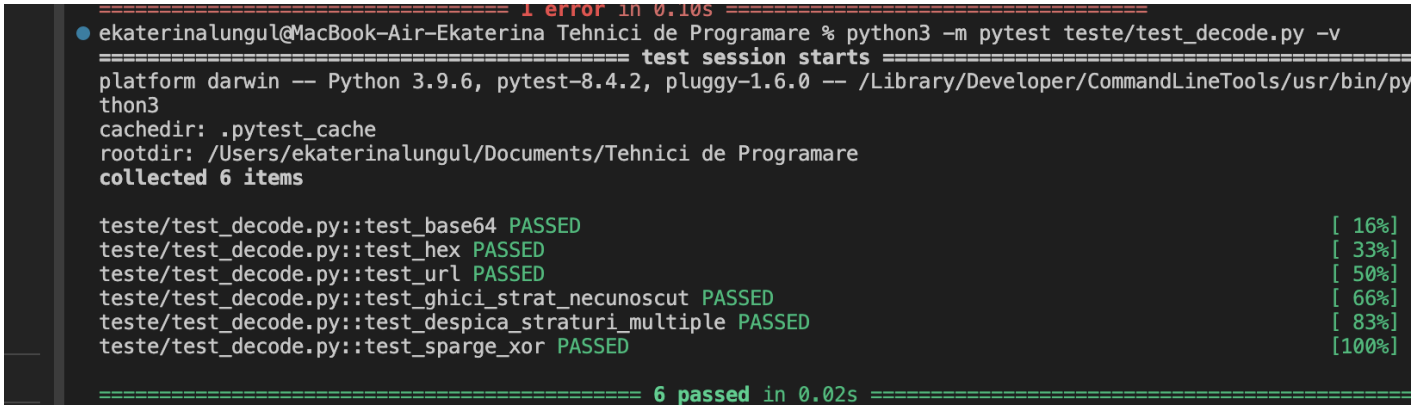


Fig.1.7 - Rezultatele testării pytest

II. Lucrarea B - Forensics pe fisiere si steganografie

A doua parte a lucrării abordează analiza fișierelor “din exterior” , fără a le deschide cu aplicația lor obișnuită , pentru a descoperi informații ascunse , precum și implementarea unui algoritm de steganografie pentru ascunderea și recuperarea unui mesaj dintr-o imagine.

2.1 Identificarea tipului real al fișierului (magic bytes)

Extensia unui fișier este doar o convenție , nu o garanție a conținutului real . Fiecare format de fișier începe însă cu o secventa fixa de octeți (semnătura , “magic bytes”): JPG începe cu FF D8, PNG cu 89 50 4E 47 , iar o arhiva ZIP cu 50 4B 03 04 (literele “PK” , inițialele lui Phil Katz, autorul formatului) .

(probe/xor.bin -> d5eea0f3e9eee7f5 (nicio semnatura cunoscuta)

```
ekaterinalungul@MacBook-Air-Ekaterina Tehnici de Programare % python3 forensics.py
probe/foto.jpg -> jpeg
probe/ascuns.png -> png
probe/xor.bin -> d5eea0f3e9eee7f5
ekaterinalungul@MacBook-Air-Ekaterina Tehnici de Programare %
```

Fig.2.8- Identificarea “magic bytes”

2.2 Extragerea textului imprimabil strings

Un fișier binar conține , de regulă , și fragmente de text imprimabil (nume de fișier , mesaje , semnături) , identificabile prin căutarea secvențelor de minim 4 caractere imprimabile consecutive.

IHDR IDATx IEND - markeri normali PNG

```
=== strings pe probe/ascuns.png ===
IHDR
IDATx
IEND
secret.txtFLAG{ascuns_prin_carving}
secret.txtPK
```

Fig.2.9 - Extragerea strings

2.3 Metadate EXIF și coordonatele GPS

Fotografiile digitale conțin, de regulă , metadate EXIF : marca și modelul aparatului , data capturii și , dacă localizarea era activa - coordonatele GPS exacte . Coordonatele GPS sunt stocate în format : grade-minute-secunde și au fost convertite în format zecimal cu formula : grade ++ minute/60 + secunde/3600.

Nota: Acest rezultat ilustrează motivul pentru care se recomanda stergerea metadatelor eXIF înainte de publicarea unei fotografii online : locul exact al capturii poate fi extras de oricine care are acces la fișiere: (tehnica utilizata in investigatiile de tip OSINT) .

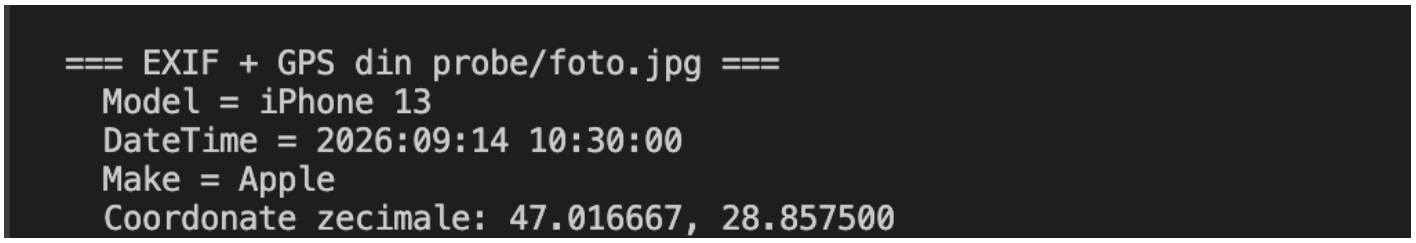


Fig.2.10 - Metadatele și coordonatele GPS identificate

2.4 File craving - extragerea unui fisier ascuns

Formatul PNG are un marker oficial de final (IEND) , insa spectrografe / analizatoare de imagini ignora orice date suplimentare adaugate pe acest marker. Cautand semnătura unei arhive ZIP oriunde în fișier (nu doar la început) se poate extrage tot ce urmează de la acel punct intr-o arhiva funcțională separată .

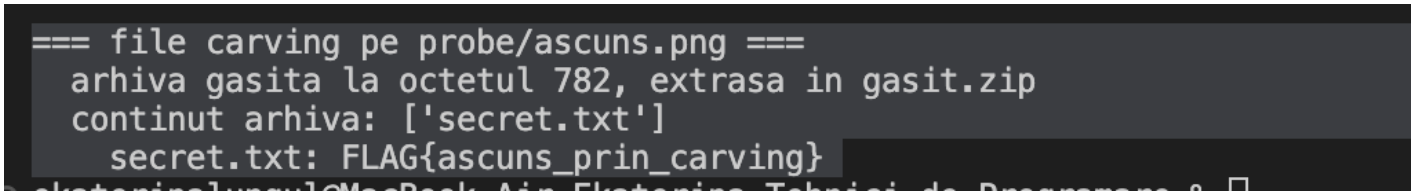


Fig.2.11- Extragerea arhivei secrete

2.5 Steganografie LSB - ascunderea și recuperarea unui mesaj

Fiecare pixel al unei imagini RGB conține trei valori (Roșu , Verde , Albastru), fiecare în intervalul 0-255. Modificarea exclusiv a ultimul bit (LSB- Least Significant Bit) al fiecărei valori schimba culoarea rezultată cu cel mult o unitate din 255 , o diferenta nesimtita vizual , dar suficientă pentru a codifica informația - bit cu bit .

Ascunderea mesajului:

mesajul text este transformat în octeți , iar fiecare octet in 8 biti ;

la final se adaugă un octet terminator (0x00) , care marchează sfârșitul mesajului la citire;

pentru fiecare cabal R,G,B al fiecărui pixel , se înlocuiește ultimul bit al valorii cu următorul bit din mesaj , folosind operația (valoarea AND-1) OR bit.

Recuperarea mesajului (operația inversa):

se citește ultimul bit al fiecărui canal R/G/B , în ordine , pentru toți pixelii ;

biții sunt regrupati câte 8 , pentru a reconstitui octeții originali ;

procesul se oprește la intalnirea octetului terminator (0x00).

Nota: Fișierul stego.png are exact aceleași dimensiuni ca foto.jpg (640x480 pixeli) și este insesizabil vizual de original - diferenta se afla exclusiv în ultimul bit al fiecărui canal de culoare .

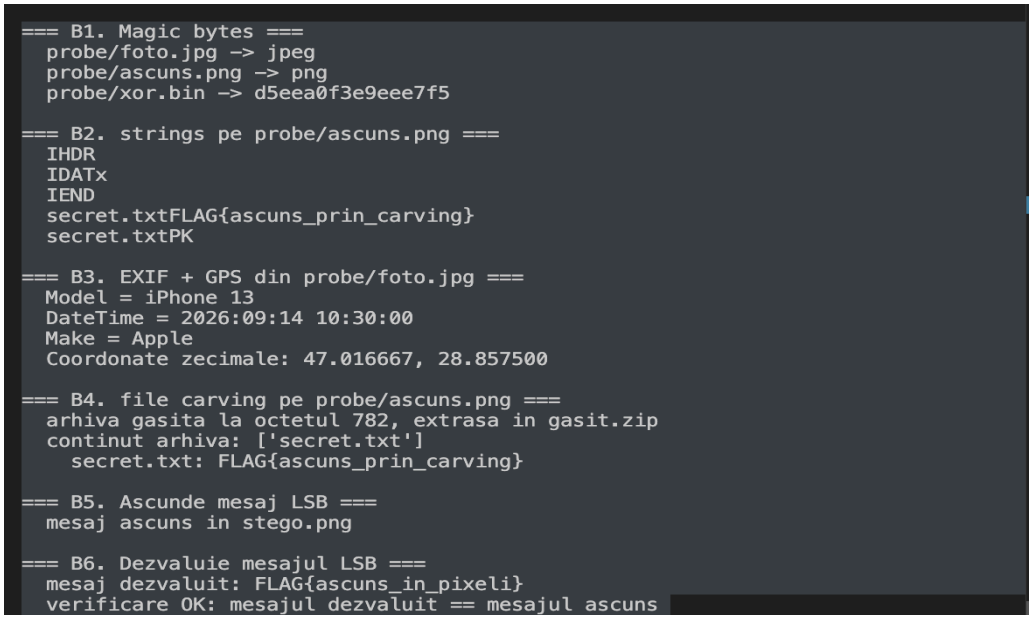


Fig.2.12 - Rezultatele finale lucrării B

III. Sarcina suplimentară - challenge bin

Indiciul asociat fișierului challenge.bin indica un lanț de codare format dintr-un strat de base64 aplicat peste o cifrare XOR cu cheia de un octet . Desfacerea s-a realizat în doi pași , în ordine inversa fără de construcție : mai intai decodarea base64 , apoi spargerea prin forța brută a cifrarii XOR rămase .

```
cheie XOR: 128 -> FLAG{lant_complet_despicat}
md5 verificare: 8c4eb02d9dcef7d0ff91d8e4c7fea91b   (identic cu indiciul)
```

Fig.3.13- Rezultatul spargerii lanțului cu cheia XOR

CENTRALIZAREA REZULTATELOR

Sursă	Rezultat obținut
mesaj.txt (decodare straturi)	FLAG{straturi_de_encoding}
xor.bin (spargere XOR)	cheia 128 -> FLAG{xor_un_octet}
hashuri.txt (spargere hash)	md5 -> parola: password
ascuns.png, strings	indiciu: secret.txt + PK
foto.jpg, EXIF/GPS	47.016667, 28.857500 (Chisinau)
ascuns.png, file carving	FLAG{ascuns_prin_carving}
stego.png, steganografie LSB	FLAG{ascuns_in_pixeli}
challenge.bin (bonus)	FLAG{lant_complet_despicat}

Fig.3.14 - Rezultatele finale a lucrării de laborator

CONCLUZII

Lucrarea de laborator nr. 6 a permis aprofundarea și cunoașterea mai detaliată a unor concepte fundamentale din domeniul securității informaționale și al analizei forensice digitale . Partea de decodare a demonstrat importanța recunoașterii corecte a tiparelor de codare și a criteriilor de oprire într-un algoritm frecventiv , evidentiind o capcana reala : un rezultat “ imprimabil” nu este , prin el insusi , o dovada ca s-a ajuns la textul final , atunci cand mai multe straturi de codare pot produce , fiecare , text imprimabil .

Partea de forensics a arătat ca informații aparent invizibile - tipul real al unui fișier , text ascuns în interiorul unui binar , metadate EXIF , fișiere lipite la coada altor fișiere sau mesaje codificate în biții cei mai puțin semnificativi ai unei imagini - pot fi descoperite sau construite cu instrumente relativ simple , scrise în cateva zeci de linii de cod Python . Toate cele opt rezultate (șapte obligatorii și unul suplimentar) au fost obtinute si verificate cu succes , iar corectitudinea decoderului a fost confirmată printr-un set de teste automate.